Code Jam 2008 - EMEA Semifinal

Analysis: Bus Stops

The problem can be solved by using dynamic programming and matrix multiplication for solving linear recursive sequences.

Consider a configuration of buses that are within a window of width P and advance the leftmost bus in such a way that all of the buses are still within a (shifted) window of width P.

Now we can define a state as the position of the buses within P units. For instance if we have P = 10 and K = 5 buses, then we have 252 possible states (10 choose 5). Let NS be the number of states.
To be sure we don't count the same state in different windows we can required that there always be a bus at the leftmost position in the window.

To advance the window of size P we move the leftmost bus. There is always a bus there. Now remember that the new state has to have a bus at the leftmost position, so the window might move to the right by more than one spot.

We compute all the possible state transitions. Let C be the matrix of state transitions having $C_{a,b}$ be the number of ways we can go from state a to state b.
For each state j, with the window in the position i we will have something like:

$$A[S_j][i+1] = C_{1,j}A[S_1][i] + C_{2,j}A[S_2][i] + \dots + C_{NS,j}A[S_{NS}][i]$$

A[s][p] is the number of ways we can get into state s while having the window in position p.

This would be enough to solve the small input. For the large input we need to speed things up, so we are going to compute that linear recurrence of order NS by using matrix multiplication. Let's look instead at a much simpler example of a linear recurrence -- Fibonacci numbers.

$$F_N = F_{N-1} + F_{N-2}$$

This can be rewritten as:

$$\begin{pmatrix}1 & 0\\ 1 & 1\end{pmatrix} * \begin{pmatrix}F_{N-2}\\ F_{N-1}\end{pmatrix} = \begin{pmatrix}F_{N-1}\\ F_N\end{pmatrix}$$

In a shorter form we have:

$$M * V_{N-1} = V_N$$

This says that to compute $V_N = (F_N, F_{N-1})$ from $V_{N-1} = (F_{N-2}, F_{N-1})$ we need to multiply with M. Now we can apply this recursively:

$$M^X * V_0 = V_X$$

M^X can be computed using an algorithm called *successive squaring* in time that is logarithmic in X.

Code Jam 2008 - EMEA Semifinal

Analysis: Painting a Fence

This was meant to be the easiest problem in the match. A straightforward brute force solution suffices.

Step 1. Pick a set **C** of (up to) three colors to be used. There are **N** different colors, so $O(N^3)$ such choices.

Step 2. From the offers, filter out the ones where the color is not in the set **C**. We want to figure out whether the remaining offers can cover the whole fence, and if so, what the minimum required number of offers is.

Step 2 is a classical problem with a greedy scanline solution. We sort the offers (intervals) by their left endpoint, and scan from left to right, considering the offers one by one. At any moment, if we have covered the fence from section 1 to **k**, we always pick the next offer so that it starts before **k** and ends as far to the right as possible.

If we sort all the offers by their left endpoing in the beginning, then step 2 takes time $O(N)$, and the whole algorithm runs in time $O(N^4)$.

Code Jam 2008 - EMEA Semifinal

Analysis: Rainbow Trees

Pick any vertex **r**. We view the tree as rooted at **r**, and -- as usual -- draw the tree with the root at the top, and draw the nodes of depth **d** at the same level, **d** units below the root.

By a *partial coloring* on a subset of edges we mean the assignment of colors to the edges in the subset so that the "rainbow coloring" constraint is satisfied on that subset. We do not care about the coloring of the edges that are not in the subset.
For each node **x**, we define the value

> f(x) := number of ways to color the subtree rooted at **x**, given any partial coloring for the set of edges that are incident to the parent of **x**.

It is not at all obvious that f(x) is well defined. (Why would the number always be the same for any given partial coloring?). To see that f(x) is indeed well defined, let's look at an algorithm for computing it.

Suppose **z** is the parent of **x**, and the degree of **z** is **D**. Note that the rainbow constraint is a very local condition -- the color of any edge other than those incident to **z** does not affect the coloring for the subtree rooted at **x**.

Assume that **x** has **t** children -- $\mathbf{y}_1$, $\mathbf{y}_2$, ..., $\mathbf{y}_t$. To color all the edges in the subtree rooted at **x**, we do the following.

- Color the edge **x** $\mathbf{y}_1$. There are **k** - **D** choices, since this edge cannot have the same color as any of the **D** edges incident to **z**, and no other edge in the partial coloring puts any constraint on it.
- Color the edge **x** $\mathbf{y}_2$. There are **k** - **D** - 1 choices, since this edge cannot have the same color as any of the **D** edges as above, nor the same color as **x** $\mathbf{y}_1$.
- ...
- Color the edge **x** $\mathbf{y}_t$. There are **k** - **D** - t + 1 choices.
- Now we have a partial coloring where all the edges incident to **x** are colored. We color the subtree rooted at $\mathbf{y}_1$. There are $f(y_1)$ ways.
- ...
- There are $f(y_t)$ ways to color the subtree rooted at $\mathbf{y}_t$.

Indeed, the computation only depends on **D**, the degree of the parent of **x**. f(x) is the product of the numbers above.

There is nothing very special about the root **r**, except that we need to agree **D** = 0 in that case. And the solution to our problem is just f(r).

Code Jam 2008 - EMEA Semifinal

Analysis: Scaled Triangle

In this problem we are given a triangle and another triangle obtained by applying an affine transformation on the first one -- translation, rotation and scaling. We are asked to find a fixed point for this transformation.

This is actually a particular case of "Bannach's fixed point theorem" which guarantees the existence and uniqueness of fixed points of certain self maps of metric spaces, though knowing the theorem was not a requirement.

To solve this problem, we need to find the parameters of the transformation.

Such transformations are familiar to anyone who has ever played with computer graphics. To view the transformation as a linear operator, it is convenient to use homogeneous coordinates, where our plane is embedded as the plane z=1 in 3D space. i.e., consider each point (x, y) as (x, y, 1). (*Note*: As usual, all the vectors corresponding to points are considered column vectors, in spite of the horizontal way we write them here.) In this setting, rotating a point by an angle alpha around the point (0, 0) corresponds to multiplying the vector v = (x, y, 1) by the matrix R = [[cos alpha sin alpha 0] [-sin alpha cos alpha 0] [0 0 1]]. Translating a point x, y by (dx, dy) corresponds to multiplying v by T = [[1 0 dx] [0 1 dy] [0 0 1]], and a scaling transform centered at 0 corresponds to multiplying a point v = (x, y, 1) by the matrix S = [[S 0 0] [0 S 0] [0 0 1]]. The total transform looks like this

```
v' = T R S v = M v.
```

Note that above we focused on the effect of the transformation on the plane z=1. The interested reader can verify that it maps every horizontal plane $z=z_0$ to itself.

To get the matrix **M**, we may solve separately for the matrices T, R, and S. There is an easier way. From the input constraints, we know that point A is mapped to A', B to B' and C to C'. View each point as a vector in the plane z=1, so that A, B, C are linearly independent. So the 3 by 3 matrix [A B C] is invertible. Therefore,

```
M [A B C] = [A' B' C']
```

has a unique solution for **M**.

From here, there are still two solutions to our problem. One observation is that if we apply the transformation and we have a point that doesn't change, we can apply it again on the resulting triangle and the point will remain the same. So we apply this transformation until the triangle becomes very small. The fixed point will still be inside the triangle so we can stop when the side lengths of the current triangle get smaller than the needed precision.

The more algebraic solution looks at the equation again. We know there is a unique point v = (x, y, 1) such that **M** v = v. From here we know that (i) 1 must be an eigen-value for the matrix M; (ii) the space of the eigen-vectors corresponding to 1 must be one-dimensional.
So we can just solve the equation [M - I] v = 0, and find the intersection of the solution space (a line) and the plane z=1.

More information

[Banach fixed point theorem](#) - [Homogeneous coordinate](#) - [Eigen value, eigenvectors, and more](#)